

修 士 論 文

Multi Agent Path Planning using Ant Colony Optimisation

蟻コロニー最適化方法を用いたマルチエージェント経路探索

東京電機大学大学院工学研究科

電気電子工学専攻電子光情報コース修士課程

18KMH18 Joly Raphaël

研究指導教員 教授 五十嵐 洋

謝 辞

I would like to thanks Professor Igarashi (Tokyo Denki University) for his guidance and encouragement throughout this research. I want to thanks as well Professor Wada for his work as a reviewer of this paper.

And for all the members of Collaborative Robotics Laboratory, I really enjoyed working with you all, thank you very much.

目 次

Introduction	1
第 1 章 Multi Agent Path Planning	2
1.1 Applications	2
1.2 The different types of MAPP	3
第 2 章 Ant Colony Optimisation	5
2.1 Definition	5
2.2 The ACO probabilistic model	6
2.3 The ACO pheromone update	6
第 3 章 Approach and Results	8
3.1 Problem Description	8
3.1.1 Graph model	8
3.1.2 Robot Model	8
3.2 The algorithm	9
3.2.1 Vehicle initialization	9
3.2.2 Shortest path calculation	10
3.2.3 Collision verification	10
3.2.4 Ant Colony Optimisation search	10
3.2.5 Vehicle addition order	12
3.2.6 Rule-based Algorithm	13
3.2.7 Evaluation	14
第 4 章 Implementation of a Multi-Agent System (MAS)	18
4.1 Agent's movement control	18
4.2 Map's layout and agent's positioning	21
4.3 Communication	23
4.4 MAPP algorithm adaptation	24

4.4.1	Task management and Action Dependency Graph (ADG)	25
4.4.2	Lifelong Planning	25
付 録 A Serial Slave Command for m3pi		28

Introduction

In my interest to learn about Artificial Intelligence (AI), I came across a lot of different applications and lectures about AI. Neural Network, Reinforcement Learning and other. After discussion with professor Igarashi, I discovered about swarm intelligence and took interest in its applications and in the different shapes the swarm intelligence could take. I focused in particular toward Particle Swarm Optimisation (PSO) and Ant Colony Optimisation (ACO). I first familiarized with the properties of these algorithms by learning the basics on the net and by programming simple applications.

I then searched for a new possible application for the swarm intelligence. I wondered if it could be used for an existing complex problem with a possibility of a real implementation and so I chose Multi Agent Path Planning (MAPP).

The benefits of MAPP are its complexity and the numerous area it can be applied on. Also, MAPP determination revolve strongly around the environment and the condition applied to the agents so MAPP is a problematic complex enough to make a 1.5 years work subject. MAPP also leaves enough freedom to choose the appropriate determination method for the environment considered.

As for the swarm intelligence algorithm applied to MAPP, I found Ant Colony Optimisation to be more appropriate for the graph type environment I wanted to use. Also, ACO is granting an easier algorithm design thanks to its intrinsic parameters which allows me to tune the ACO to do more in-depth searches. These benefits of the ACO are more discussed in the ACO chapter.

第1章 Multi Agent Path Planning

Multi-Agent Path Planning (MAPP) is a well-researched problem in heuristic programming or Artificial Intelligence (AI) area. The idea of MAPP is to find a path for every agent of a network such that no collision occurs between them by considering each agent's position, their goal and the map topography. An example of MAPP problem is shown on Fig. 1.1.

1.1 Applications

As I mentioned, MAPP determination depends greatly on the environment considered. Whether virtual, indoor, outdoor, 2D, 3D, free or intricate space, a MAPP solution designed for one of these environments may not be used for an other environment.

For example, between a virtual and real environment, problematics will be different as it is possible to ignore an agent size or to consider some action as instantaneous in a virtual environment. The height of an agent is ignored for a 2D environment but may be considered for other type of environment.

That's why for all the research about MAPP that I could read, it was important to consider what kind of environment was used and what limitation or freedom it would involve.

As for my research, I focused on a real, indoor, 2D environment for warehouse logistic application.

Application for MAPP can be found in computer games, airport scheduling or warehouse automation [5].

In 2016, \$ 1.9 billion has been invested in warehouses automation and this number is increasing (<https://www.tractica.com/research/warehousing-and-logistics-robots/>). This shows a need from the industry for automation technology and for a fitting methods of path planning.

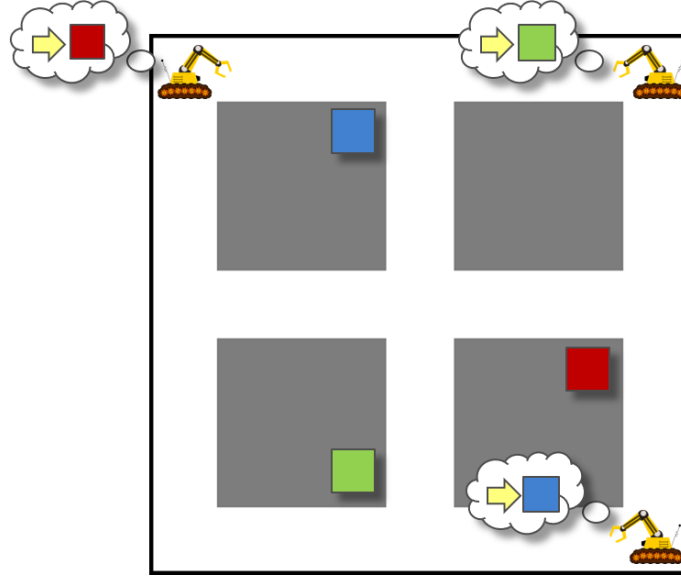


Fig. 1.1: Example of a MAPP problem

1.2 The different types of MAPP

Multi-Agent Path Planning (MAPP) is a NP-hard problem. It is formulated as follow: Given an unweighted undirected graph of the environment and a set of agents with start and goal location, determine a collision-free path for each agent. Agents initially reside in their start location and must eventually reach their goal location.

Existing methods for MAPP are search-based, reduction-based or rule-based [6]. Search-based methods will calculate agent's paths one at a time, any agent searching for a path may not occupy specific locations at specific times reserved by previous agents. Rules-based methods consist on imposing rule of movement for every agent based on pattern on the map or any other circumstances. An example can be seen in [4] where each agent has its own movement rules depending on previous environment states. Reduction-based methods consist on reducing the MAPP problem to a standard known problem such as the Boolean satisfiability problem (SAT) for which the solvability and efficiency have already being proven.

My approach is a search-based method as I am using the results of other agents to calculate a new path. However, we will see that calculating the path of agents one by one poses a problem of mutual optimality as the first agent for which the path is calculated

will usually have a shorter distance to travel compared to the later agents. That's why I also considered using a rule-based approach in the development of my algorithm.

第2章 Ant Colony Optimisation

ACO is an Artificial Intelligence (AI) based on biological inspiration of real ant colonies. Since its discovery, this AI has found many uses and improvement in its algorithm [1][2]. Most of its application are for problem with deep search of the environment and distribution [3] or multi-objective optimization. This is why I think it's a suitable AI for developing a MAPP solver.

Research [3] is about using ACO to create a new driving assistance that is avoiding congestion by flow distribution on roads. Its results motivated me to use ACO for not only assistance but the whole planning system with the deep search and vehicle distribution possibilities of the ACO being the core of search-based method of MAPP.

2.1 Definition

ACO is an optimisation method using a population of artificial agent, the "ants". Agents build solutions by moving on a graph-based representation of the problem. The movement of the ants on the graph is probabilistic and depends on a parameter that will evolve throughout the search so as to increase the probability that future ants will build better solutions. This parameter is referred to as "pheromone".

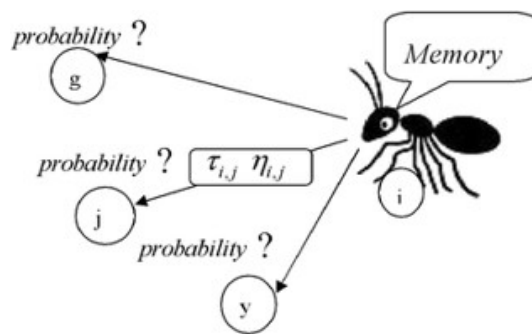


Fig. 2.1: Decision making of an ant from ACO

2.2 The ACO probabilistic model

When at an intersection x , the probability for an ant to go from x to intersection y is the following:

$$p_{xy} = \frac{\tau_{xy}^{\alpha} \sigma_{xy}^{\beta}}{\sum_z \tau_{xy}^{\alpha} \sigma_{xy}^{\beta}} \quad (2.1)$$

With τ_{xy} being the pheromone value. σ_{xy} is an "affinity" function. z corresponds to all the other intersections connected to x and (α, β) coefficients attached to pheromone and affinity respectively. The affinity is not always used and is usually depending on the path length. But other affinity function can be implemented. As this affinity doesn't depend on pheromone value it is used for orientating further the search.

2.3 The ACO pheromone update

After every ant search, a pheromone update is performed depending on how good the ant's path is. The update equation is the following:

$$\tau_{n+1}(i, j) = (1 - \rho) \cdot \tau_n(i, j) + Q \quad (2.2)$$

With $\tau_n(i, j)$ being the pheromone value of edge connecting intersections i and j at step n . ρ is the evaporation rate and Q is the quality function depending on whether the ant took this edge or not and on how good the ant performed. Evaporation is important to reduce the pheromone on unused paths.

Several other strategy have been introduced to the pheromone update like iteration best update or global best update where the pheromone are updated only considering the best found path of a series of ant search or the overall best found path respectively.

Another modification of the generic update was introduced in MAX-MIN Ant System [1], which uses maximum and minimum pheromone edge limits. With this modification, the probability of generating any particular solution is kept above some positive

threshold. This helps preventing search stagnation and premature convergence to sub optimal solutions

第3章 Approach and Results

3.1 Problem Description

3.1.1 Graph model

I am considering the map of a warehouse which I transform into a graph. Nodes of the graph are placed on crossroads with 3 or more paths connected to it. I consider that connections are too narrow for two agents to be next to each other or manoeuvre around each other. This does imply that there cannot be any agents that are taking the same connection on opposite direction at the same time. This is the case for warehouses with robots following a line to move inside their facility.

At the start of the simulation, agents are all placed on different nodes and must get to another node of the graph. For realistic purpose, I gave nodes a proper size and added the rule: “There can be only one agent on the node at a time”.

3.1.2 Robot Model

In my model, all robots have the same behaviour while moving and turning. Their speed is fixed while moving from a node to another and the time it takes for decelerating, turning and accelerating is considered constant. These parameters are used in the ACO calculation and more precisely in the timing consideration. This does mean that, if a better model is implemented, it can be integrated by the ACO algorithm.

As a future work, a model that is representing the vehicle used in a real scenario could be implemented. For this, I need to compare timing of the real experimentation with my intern timing and adjust the model's parameters.

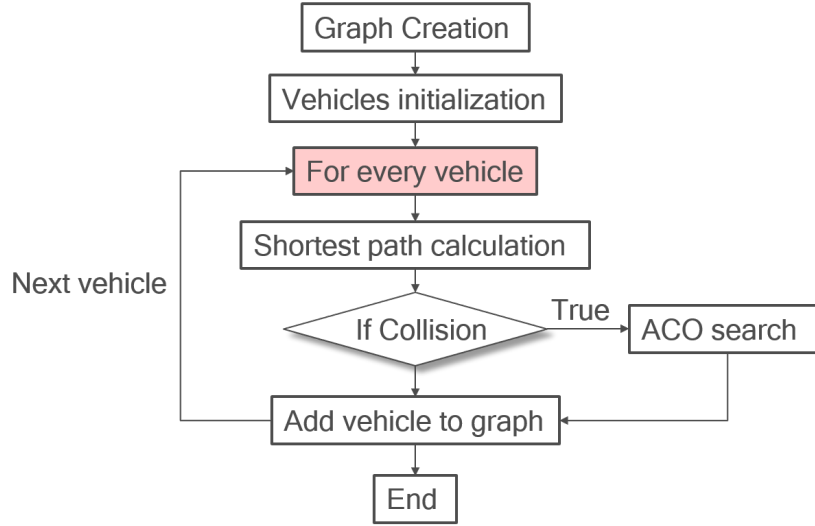


Fig. 3.1: MAPP determination's flow chart

3.2 The algorithm

The path calculation for every agent is done as it is describe in the flow chart on Fig. 3.1.

3.2.1 Vehicle initialization

The entire programming has been using Object Oriented Programming on Python 3.6. The objects representing the vehicles are created before the calculation and have the following variables.

- “Start ” : the number of the node the vehicle is starting on.
- “Destination ” : the number of the node the vehicle is heading to.
- “Graph ” : the graph on which the vehicle is moving. To make calculation more effective, this graph is the original one with all dead-end removed which are not the starting node nor the destination node.
- “Path ” : the path left to be travelled by the vehicle. It is a list of numbers corresponding to the nodes the vehicle will be crossing.

- “Path division”: a dictionary type data. It regroups the different edges the vehicle is crossing during its travel and the timing associated with them. As it is equivalent to wait before a node and crossing an edge slower, I didn’t include waiting timing in the “path division”. This dictionary is also used to verify any collision during or after calculation.

3.2.2 Shortest path calculation

In order to get a first optimal path for the vehicle and also to not let the ACO wander randomly at the start of the calculation, I first calculate a single agent optimal path for the vehicles using Dijkstra algorithm.

This path calculation has two uses. Firstly, it is the most preferable case if the vehicle can use its most optimal path without colliding with the other agents. Secondly, the ACO is an algorithm that is searching for a solution randomly so this path is used to focus the search around it.

3.2.3 Collision verification

A verification of whether there is a collision or not is used throughout the path calculation. It is first used after the single-agent shortest path calculation to verify whether this path is implementable with the vehicles that are already on the map. It is also used after the ACO search to determine the success rate, a value that is used for the evaluation.

3.2.4 Ant Colony Optimisation search

As it can be seen in the flow chart, the search using the ACO is only used if a collision is detected between considered vehicle and those already on the map (who already have a calculated path). This allow for a quicker calculation when no problem is detected beforehand or for solo-task.

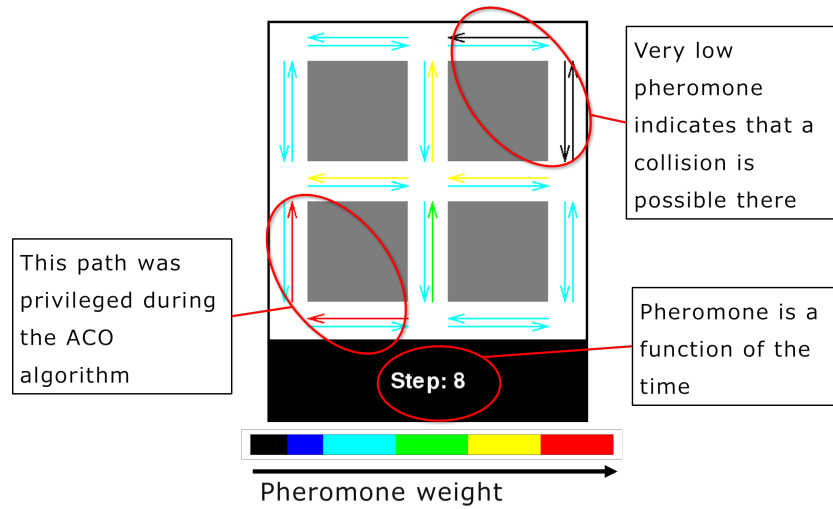


Fig. 3.2: Example of pheromone map on step 8 of the ACO search.

Ant Colony Optimisation algorithm basically works using simple elements named the “ants” that will search the environment until it reaches its goal. When moving, the ants will choose randomly between different paths and it will have a higher chance of choosing a path with a high amount of pheromone on it. When the ant reaches its goal, it put pheromone on its path. The amount of pheromone the ant is putting depends on how well it performed. Usually, the shorter the path the ant took is, the higher is the amount of pheromone. See chapter 1.2 for more detailed information on the mathematical implementation.

In my algorithm, the ACO search is done in 5 parts:

- Creation the “Pheromone map” object. This object contains the value of pheromone of every edge at any time of the simulation. An example of pheromone map can be seen in Fig. 3.2. On this example, the colour of the arrows corresponds to the weight of pheromone on it. It can be noticed that a direction and its opposite are considered differently on an edge. Also, as it has been explained in [1], I defined a maximum and minimum value for the pheromone. I ensured that for any situation, the minimum probability for an ant to take a path is 5%.
- Apply bonus pheromone: Considering the path calculated via Dijkstra algorithm, the connection following the path or leading to it see their pheromone raise by a

constant bonus.

- Apply malus pheromone: Considering the path of the other vehicles already added to the graph, every edge where a collision can happen see its pheromone reduced to zero, ensuring that the ants will not take that path.
- ACO algorithm: waves of ants will search for the objective. When it arrives at a node, an ant will choose between a set of connection randomly with the restriction that it cannot choose the connection that it just took before to avoid an inefficient U-turn. If all the pheromone of the connections around are null, the ant waits. The number of waves and ants per wave is arbitrary for the moment. These parameters can be raised or lowered depending on the performance needed: calculation time or optimality.
- Path choice using the pheromone map. When the ACO algorithm is over, The ACO search return a path for the vehicle that is calculated using the pheromone map. For now, the path with the most pheromone is chosen at every step but other methods can be found to optimise another parameters such as waiting time or road usage.

3.2.5 Vehicle addition order

What also have an importance is the order in which the vehicles are added to the graph. In my experiments, the order is arbitrary but this can cause situations where vehicles cannot avoid collision on maps with dead ends or 1-connected graphs. In Fig. 3.3 we can see an example of a situation leading to an unavoidable collision caused by a wrong calculation order.

To avoid this situation one possibility could be to use maps with no dead-ends or at least 2-connected graphs. But this solution is not applicable to all warehouses' layout. A second solution would be to choose an order of calculation beforehand or when an unavoidable situation is detected. This solution works in most cases but the calculation is quite heavier due to the number of possibilities that must be tested.

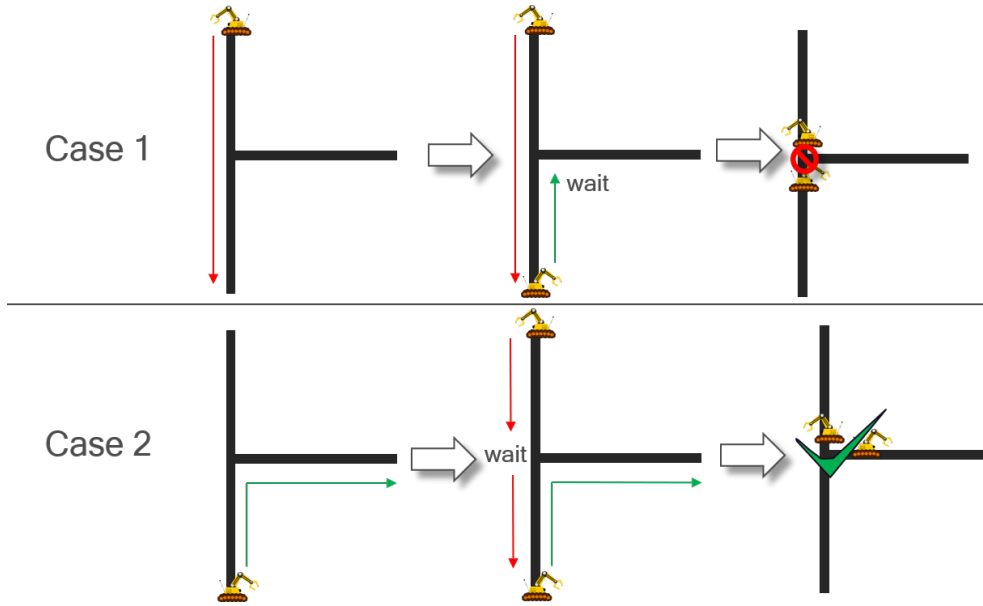


Fig. 3.3: Example of an unsolvable scenario induced by calculation order

The last solution would be to calculate the path of all vehicles at the same time and imposing crossing rule for them and thus changing the type of method from a search-based one to a rule-based/hybrid one.

3.2.6 Rule-based Algorithm

To avoid the problematic of agents calculation order, I also implemented a ruled-based algorithm using ACO. The rule-based algorithm approach is to calculate the path of all the agents at the same time. The agents next movement is imposed by a list of rules sorted by importance.

The methods I came up with for a MAPP determination using rule-based algorithm is as follow:

- At the start, every agents is choosing a path considering the neighbours agent.
- Step by step, the agents advance and the pheromones are updated to prevent the other agents to go the opposite sides.
- When an agent reach an intersection, it can either: cross the intersection if no other

agent is there or stop to let an other agent cross.

- If two or more agent arrives at the same intersection, a passage order is decided: All agents choose a path and the ones that do not cross the other agents pass in priority.
- If the intersection is blocked by too many agents, one of them do a U-turn and a passage order is decided for the remaining agents.
- When all the agents have reach their goal or when the time limit have been reached, the pheromone map of all the agents is updated considering the average traveling time. If an agent hasn't reach its goal, a malus is applied instead of considering traveling time of this agent as maximum to avoid a too negative impact on the other agents.

This algorithm is expected to have a better success rate than the search-based algorithm because it calculate the paths of the agents altogether (See evaluation section on why this is required). Also, it would also improve average traveling time of all the agents as the pheromone would be updated considering the overall travel time instead of the solo travel time like in the search-based algorithm. However, the computation time would be impacted as it will be much more difficult to converge quickly to an optimal solution. Indeed, the random path choice performed by the ACO makes the probability for all the agent to take the right path low. That's why an important work for this algorithm would be to tune the ACO's parameters to ensure a fast and robust convergence.

3.2.7 Evaluation

The search based algorithm proposed have been evaluated on two points: computation time and success rate. Computation time corresponds to how much time it takes to add all the vehicles to the graph including of course the ACO search but without taking into account graph's and vehicles' initialization (refer to Fig. 3.1). Success rate simply corresponds to whether there is still a collision at the end of the paths calculation.

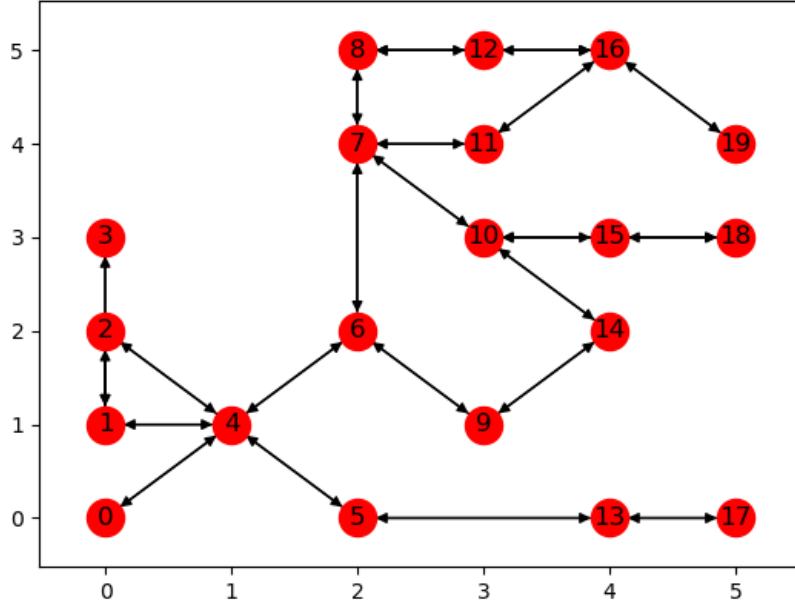


Fig. 3.4: Example of a randomly generated 20 nodes graph

To experiment on as much different graphs as possible, I created a function returning a random graph of a given size. An example of a randomly generated 20 nodes graph can be seen on Fig. 3.4.

The random graph is generated using the Python "Networkx" library's function *soft random geometric graph*. This function takes a list of nodes and will connect two nodes randomly following an inverse exponential probability law considering the distance between the nodes. The list of nodes I am using are nodes positionned randomly on a grid of size $M \times M$, M being $\{M \in \mathbb{N} | M^2 > N\}$ with N being the number of nodes in the graph and chosen beforehand. I then randomly erase nodes until the wanted graph size is reached. Then I am assuring that the graph is 1-connected (N-connected graph: when every node of a graph is connected to every other node of the graph by at least N path) by connecting unattached groups of nodes.

One hundred experiments were performed per graph size. For each experimentation a random graph was created and scenarios with 2 to 10 vehicles were tested. For each

scenarios, vehicles are placed randomly on the graph while ensuring none of them start on the same node. The destination for each vehicle is also randomly chosen. I varied graph size from 20 to 60 nodes (numbers choice is arbitrary) which makes a total of 4500 experiments.

I analysed the data using the Python's library *pandas*, allowing me to create tables of data and calculating statistics. The figures were then traced manually using Microsoft Excel. Results of the experiments are shown in Fig. 3.5 and Fig. 3.6.

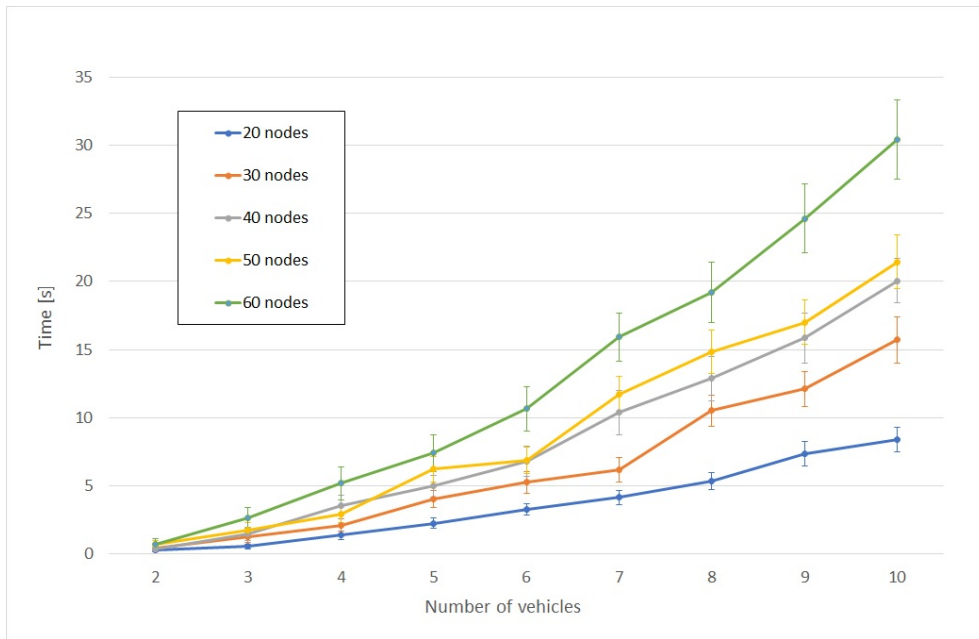


Fig. 3.5: Computation time considering the number of agent and graph size

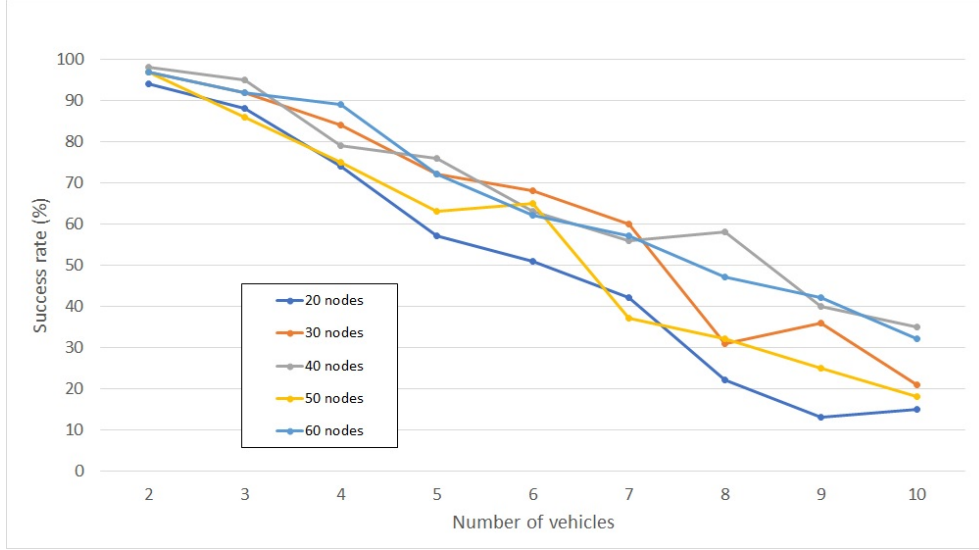


Fig. 3.6: Computation time considering the number of agent and graph size

The results for the computation time seem to follow a slow power trend which is usually the case for AI application. More experimentations are needed to compare these results with existing MAPP solver for the same scenarios.

As for the success rate, it decreases steadily with the number of agent/vehicle. This is the effect of unsolvable scenario induced by order calculation discussed in section 3.2.5.

Also, the graph of choice of this evaluation part are really the worst case scenario for this kind of search-based algorithm due to the 1-connected graph environment. 2-connected graphs would be much more performing in term of success rate. However, 1-connected graphs are what approach warehouse's layout the best in the considered context, so further evaluations with 2-connected graphs are not required in my opinion.

For future work on this research, the same evaluation should be performed on the rule-based algorithm as well as already existing MAPP method to determine its efficiency once the robustness is fixed.

第4章 Implementation of a Multi-Agent System (MAS)

A big part of my work also involved implementing a MAS and thinking about of all the problematics a real implementation has. The implementation of the MAS involved 4 areas of work:

- Agent's movement control
- Map's layout and agent's positioning
- Communication
- MAPP algorithm adaptation

4.1 Agent's movement control

To create a MAS, I used the programmable robot "m3pi" from pololu [7]. This robot is 5 cm high for a diameter of 14 cm and it weights around 180g so it's a very light robot. It is equipped with two DC gearmotors connected to two 32 mm diameter wheels. The balanced is assured by a front idler which design allows the robot to rotate on himself. The robot is controlled through an mbed LPC1768 type microcontroller.

The particularity of the m3pi is that it has 2 layers of commands: The first layer is controlled by an ATmega328 AVR microcontroller which is connected to the mbed microcontroller on the 2nd layer by a Serial port. The control of the robot's 1st layer's equipment is done undirectly by sending and receiving serial data via preprogrammed commands. For example, the command *M1 forward* which activates the first gearmotor to go forward is done by sending the command byte *0xC1* followed by one byte of data (between 0 and 127) representing the gearmotor power. A detailed list can be found in appendix Fig. A.1.

The serial control simplifies the handling and the programming of the robot, however it also limits the control possibilities for the user because some features are not accessible in the base preprogrammed serial commands. For example, we don't have access to the exact voltage passing through the motors or the different states of the buzzer for the sound.

It is possible to add a serial function manually with AVR programming, however it is not recommended to do so because losing power during programming can permanently disable the the m3pi. So it is not a tool I used.

For the m3pi movement control, I mainly used the gearmotors to move the robot forward or backward, the light sensors for line following (see section 4.2 *Map's layout and agent's positioning*) and the LCD screen for debugging and displaying the robot's different states.

For my application I needed to control the gearmotors to advance a certain distance, turn a certain angle and follow a black line under the robot at a certain speed. As I mentioned before, the gearmotors are only speed controllable via the serial command so I needed further data to do a position control. That's why I performed an interpolation's parameter determination via video tracking. The method is the following :

- Modelisation of my robot's movement: I make the hypothesis that the traveled distance or the turned angle by the robot is a polynomial function of the input function of the motors.
- I used a door function as the input function for the angle determination and trapeze function for the advance experiment.
- With constant max power, accelerating time and decelerating time and variable advancing time, I took videos and fetched the traveled distance and turned angle with a video tracking program.
- With the fetched data, I determine the parameter of the polynomial function via linear regression.

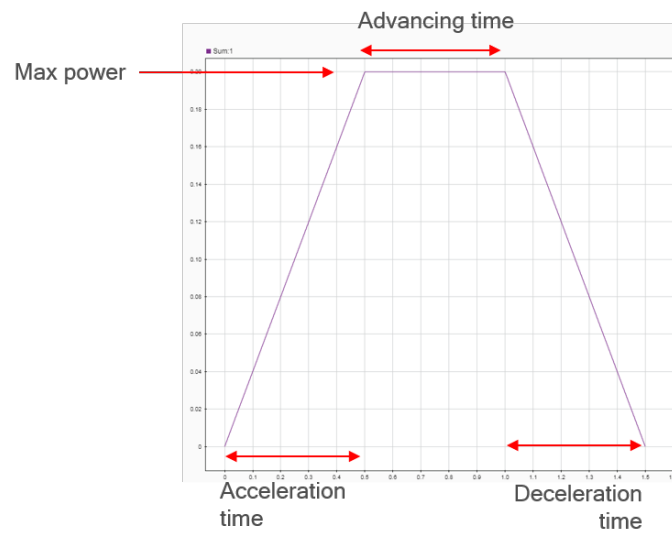


Fig. 4.1: Trapeze input function of the gearmotors



Fig. 4.2: Data acquisition for motion control of the robot via video tracking

The reason why I used a trapeze input for the distance analysis is because a steep deceleration made the m3pi stop too abruptly, sometimes going backward or turning weird angle. A linear acceleration and deceleration created smoother movements. This method gave a control precise enough for my application.

The line following function was already included in the serial command with a PID control. Using the light sensors, the m3pi is measuring its distance with the black line while moving and control the gearmotors accordingly.

4.2 Map's layout and agent's positioning

As I already have a functionality on my robots that allow them to travel along a black line, I could simplify the creation of the map layout. So I created a map with with black tape for the robot to follow. This allows me to create any type of graph and also assure the position of the robot. Then, in order to create intersections, I used tapes of different color than black so the robots can detect whether they are on black tape or on tape of different colors using their light sensors. An example of a graph created with tape can be seen in Fig. 4.3.

To differentiate colors with the robot's light sensors, I needed to get more detailed information about these sensors. These information could be found on the pololu website [8]. The light sensors the robot is equipped with are IR light sensors module composed of one IR light emitter and one phototransistor as the receiver. These modules can detect the reflectance of the surface the robot is on by emitting a flash of IR light and analysing the received light energy with the phototransistor. The sensor module then sends a number proportionnal to the surface reflectance. The higher this number is, the lower is the surface reflectance.

To get these reflectance numbers, I can use the *raw sensors* function of the serial command by sending the *0x86* command byte and it will send me a 10 bytes message containing the reflectance number of all 5 light sensor modules. I then analysed these numbers with tapes of different colors (see Fig. 4.4)

I then decided about thresholds separating the black tape from the gray tape. We can

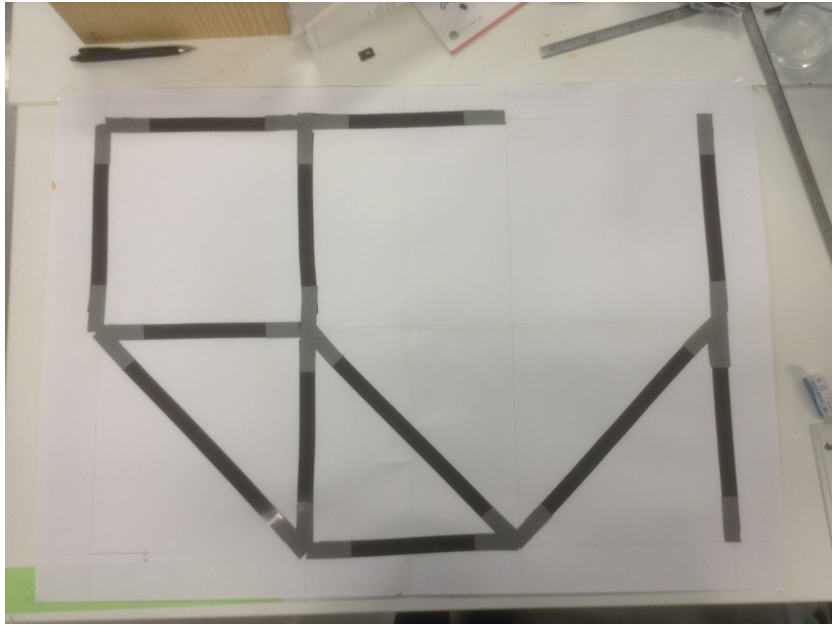


Fig. 4.3: Graph created with Black and Gray tapes

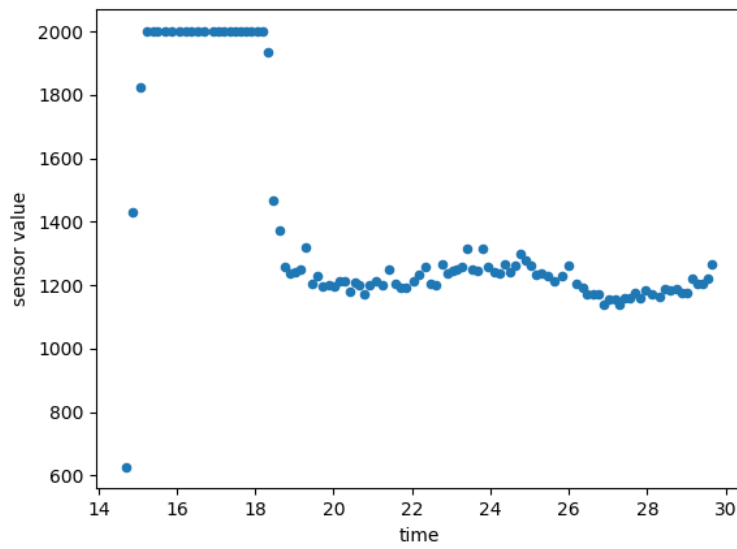


Fig. 4.4: Analysis of light sensor data with the robot moving from black to gray tape.

also see that the first data taken by the sensor are wrong: the signal slowly raise to 2000. So these wrong data must be ignored.

While moving around the map, the agents are doing as follow:

- Follow the line with the integrated line following program.
- When the gray line is detected, the agent communicate to the server that it arrived at an intersection.
- To go across an intersection, the agent first calculate the angle it needs to turn to follow its path. The angle calculation is done by integrating all intersections's coordinate in the robot.
- The agent then advances to the center of the intersection, turns the calculated angle and then restart advancing while following the tape.

As the advance and turn function as well as the gray tape detection are not perfect, the path following for the agent can sometimes fail. The most common problems I had were

- The agent detected the gray tape too early and then failed crossing the intersection
- The agent arrived at the intersection misaligned with the center of the intersection and then also failed crossing the intersection.
- The agent detects a gray line immediately after leaving an intersection and thus breaks the path sequence.

I succeeded in fixing most of these problems but it was still not 100% successful. To make my method more robust, a possibility would be to use a speed or position detector of the wheels in order to pinpoint more accurately the movement of the robot on the intersection.

4.3 Communication

In MAPP implementation, communication is important because MAPP algorithm needs information about the agents before or during the travel. There are two types of

communication for MAS: centralised and decentralised one.

- A decentralized communication for MAS is when each agent is equipped with a calculation core that will calculate the MAPP solution and then communicate their position and paths to all the other agents. In this case, all agents are independant and act accordingly to the other agents around them.
- A centralized communication for MAS is when all the data gathering and processing is done by a central computer placed outside the travelling space of the agent. The computer receive data from agents and send the adapted results of the MAPP algorithm to every agent.

For calculation performance purpose, I used a centralised communication system based on wireless Xbee module with the API mode. The API mode allows me to send serial messages with source and destination indicator for each of the messages. With this, the agents and the central computer will only read the message for them and discard the other. The central computer can then send message to one specific agent and can also determine from which agent the message it received was.

I then created different commands for both agents and central computer to send and receive specific data. The main difficulty was the management of the API mode. As the central PC uses a Python library for Xbee API mode and the agents are using an mbed library (C++ language), the message processing is different on both devices. It took a long time to adapt the code for the communication commands. For example, it is easier on python to process a whole string data whether than a single character, but it is the opposite on mbed.

4.4 MAPP algorithm adaptation

This section is entirely based on adapting the research of Hönig, Wolfgang, et al. [5] to my MAPP environment. I adapted this research in order to use the results provided by my MAPP algorithm. This allows for a simpler synchronisation of the agents and a live management of intersection crossing.

4.4.1 Task management and Action Dependency Graph (ADG)

The first part of this research is to divide actions from the agents into tasks which are logically connected. The goal is to create a ADG where all the tasks of all agents are displayed and where every connections between task represent a dependency. A task depends on another if the first task needs to be completed before the other task, creating a logical sequence of tasks. There are two sorts of dependency, the one describing an agent's own task sequence and the dependency coming from another agent.

For my application, I divided the tasks into: crossing intersection "i" and going from intersection "a" to "b". i, a and b depending on the considered graph. Then, using the time step of my MAPP algorithm, I decide on dependency between all the tasks of all agents.

During the traveling, the tasks of the ADG are divided in 3 types: *finished*, *enqueued* and *staged*.

- At the start, every tasks are *staged* except for the first tasks of agents which are *enqueued*.
- When an agent performs a task, it sends a notification to the central computer who will then mark this task as *finished*. This is also why it is important to have a communication on API where the sender and the destination are known and not just a broadcast communication.
- If the next task of the agent is *staged*, the agent is stopped and put on standby.
- When the central computer receives a notification, it then check all the tasks and *enqueued* the one that only depends on *finished* tasks. If an agent was waiting for this task to be *enqueued*, a notification to continue is sent to it.

4.4.2 Lifelong Planning

Another objective of this research is to allow for a lifelong planning. In other words, the algorithm must be able to calculate new action so that agents never stop performing

actions. Also, the algorithm must be able to calculate a new solution when a condition is no longer respected. A new obstacle for example.

For this problem, Hönig, Wolfgang, et al.'s research build the following strategy:

- In case of recalculation, the algorithm determine all tasks impacted by the new conditions.
- Every of these tasks and the tasks beyond them are erased.
- The recalculation is done starting from the remaining tasks of the ADG with the particularity that there cannot be any dependency between tasks from the old ADG (before recalculation) and the new ADG.

参考文献

- [1] Stützle, Thomas, and Holger H. Hoos. "MAX-MIN ant system." *Future generation computer systems* 16.8 (2000): 889-914.
- [2] Dorigo, Marco, and Mauro Birattari. *Ant colony optimization*. Springer US, 2010.
- [3] Stolcis, Christian, and Elmar Pfannerstill. "Clustered swarm: a live swarm-based traffic load balancing algorithm against traffic jams." *IET Intelligent Transport Systems* 11.3 (2016): 134-141.
- [4] Li, Howard, Simon X. Yang, and Mae L. Seto. "Neural-network-based path planning for a multirobot system with moving obstacles." *IEEE Transactions on Systems, Man, and Cybernetics, Part C (Applications and Reviews)* 39.4 (2009): 410-419.
- [5] Hönig, Wolfgang, et al. "Persistent and robust execution of mapf schedules in warehouses." *IEEE Robotics and Automation Letters* 4.2 (2019): 1125-1131.
- [6] Felner, Ariel, et al. "Search-based optimal solvers for the multi-agent pathfinding problem: Summary and challenges." *Tenth Annual Symposium on Combinatorial Search*. 2017.
- [7] [<https://www.pololu.com/category/111/m3pi-robot>]
- [8] [<https://www.pololu.com/docs/0J21/5.d>]

付 録 A Serial Slave Command for m3pi

Command byte	Command	Data bytes	Response bytes	Description
0x81	<i>signature</i>	0	6	Sends the slave name and code version, e.g. "3pi1.0". This command also sets motor speeds to 0 and stops PID line following, if active, so it is useful as an initialization command.
0x86	<i>raw sensors</i>	0	10	Reads all five IR sensors and sends the raw values as a sequence of two-byte ints, in the range 0-2000
0x87	<i>calibrated sensors</i>	0	10	Reads all five IR sensors and sends calibrated values as a sequence of two-byte ints, in the range 0-1000
0xB0	<i>trimpot</i>	0	2	Sends the voltage output of the trimpot as a two-byte int, in the range 0-1023
0xB1	<i>battery millivolts</i>	0	2	Sends the battery voltage of the 3pi in mV, as a two-byte int
0xB3	<i>play music</i>	2-101	0	Plays a tune specified by a string of musical commands. The first data byte specifies the length of the following string (max length 100), so that the slave program knows how many more data bytes to read. See the play() command in Section 3 of the Pololu AVR Library Command Reference for a description of the musical command format.
0xB4	<i>calibrate</i>	0	10	Performs one round of calibration on the sensors, reads all five IR sensors, and sends calibrated values as a sequence of two-byte ints, in the range 0-1000. This should be called multiple times, as the robot moves over a range from white to black.
0xB5	<i>reset calibration</i>	0	0	Resets the calibration. This should always be used when connecting to a slave, in case the master reset without a slave reset, for example in case of a power glitch.
0xB6	<i>line position</i>	0	2	Reads all five IR sensors using calibrated values and estimates the position of a black line under the robot. The value, which is sent back as a two-byte integer, is 0 when the line is under sensor PC0 or farther to the left, 1000 when the line is directly under sensor PC1, up to 4000 when it is under sensor PC4 or farther to the right. See Section 19 of the Pololu AVR Library Command Reference for the formula used to estimate position.
0xB7	<i>clear LCD</i>	0	0	Clears the LCD screen on the 3pi.
0xB8	<i>print</i>	2-9	0	Prints 1-8 characters to the LCD. The first byte is the length of the following string of characters, as with the <i>play</i> command above.
0xB9	<i>LCD goto xy</i>	2	0	Moves the LCD cursor to x-y coordinates given by the next two bytes.
0xBA	<i>autocalibrate</i>	0	1	Turns the robot left and right while calibrating. For use when the robot is positioned over a line. Returns the character 'c' when complete.
0xBB	<i>start PID</i>	5	0	Sets up PID parameters and begins line following. The first data byte sets the maximum motor speed. The next four bytes, a, b, c, and d, represent the PID parameters. Specifically, the difference in the motor speeds will be set to $(L-2000) \times a/b + D \times c/d$, where L is the position of the line as described above, and D is the derivative of L. The integral term is not implemented in this program. See Section 7.c for more information on PID line following.
0xBC	<i>stop PID</i>	0	0	Stops PID line following, setting motor speeds to 0.
0xC1	<i>M1 forward</i>	1	0	Sets motor M1 turning forward with a speed of 0 (off) up to 127 (full speed).
0xC2	<i>M1 backward</i>	1	0	Sets motor M1 turning backward with a speed of 0 (off) up to 127 (full reverse).
0xC5	<i>M2 forward</i>	1	0	Sets motor M2 turning forward with a speed of 0 (off) up to 127 (full speed).
0xC6	<i>M2 backward</i>	1	0	Sets motor M2 turning backward with a speed of 0 (off) up to 127 (full reverse).

Fig. A.1: List of base serial slave program of the pololu m3pi